Software Engineering Lab

Lab 18

Deadlock di Sistem Konkuren

Mekanisme Siklus Tunggu, Penanganan Database,
dan Strategi Aplikasi di Go



Lukman / lukman-ss

Lonjakan Transaksi Konkuren Memicu Kemacetan Sistemik

Ketika dua proses mengunci data yang sama dengan urutan berlawanan, sistem terhenti total.

DAMPAK DEADLOCK PADA SISTEM TRANSAKSIONAL

Peningkatan Latensi

Transaksi tertahan tanpa batas waktu hingga mekanisme deteksi siklus basis data terpicu.

DAMPAK DEADLOCK PADA SISTEM TRANSAKSIONAL

Gugurnya Transaksi

Database membatalkan satu transaksi sepihak; klien menerima galat pembatalan mendadak.

DAMPAK DEADLOCK PADA SISTEM TRANSAKSIONAL

Bukan Berarti Rusak

Deadlock bukan berarti basis data rusak, melainkan akibat pola urutan akuisisi lock aplikasi yang acak.

Ketika volume transaksi tinggi mengakses baris data secara bersamaan, urutan akses yang tidak teratur memicu kebuntuan fatal. Munculnya galat pembatalan transaksi bukan berarti database mengalami kerusakan internal, melainkan sinyal bahwa lapisan aplikasi membiarkan pola penguncian saling silang.

Uncoordinated concurrent resource locks inevitably produce circular dependencies under sustained traffic.

Review Kode Manual Gagal Menangkap Siklus Tunggu Konkuren

Mengandalkan insting tanpa standarisasi urutan lock membiarkan celah deadlock lolos ke produksi.

KELEMAHAN PENDEKATAN NAIF PENGUJIAN

Interleaving Gaib

Jadwal eksekusi thread bergantung pada thread scheduler runtime dan OS, sulit direproduksi lokal.

KELEMAHAN PENDEKATAN NAIF PENGUJIAN

Unit Test Tunggal Lolos

Pengujian fungsional sekuensial selalu berhasil karena kompetisi sumber daya tidak pernah terpicu.

KELEMAHAN PENDEKATAN NAIF PENGUJIAN

Tidak Otomatis Aman

Membungkus transaksi dengan blok ACID tidak otomatis aman dari kebuntuan bila urutan query berbeda.

Review manual dan unit test sekuensial tidak mampu mereproduksi deadlock karena eksekusi konkuren bergantung pada timing scheduler CPU. Menjalankan query di dalam transaksi ACID tidak otomatis melindungi sistem dari deadlock jika dua alur bisnis mengunci entitas dalam urutan berlawanan.

Sequential testing cannot reveal non-deterministic timing windows in concurrent resource acquisition.

Anatomi Siklus Tunggu: Deadly Embrace Dua Transaksi

Empat syarat Coffman terpenuhi secara simultan hingga membentuk dependensi melingkar.

ROW EXCLUSIVE

1. Mutual Exclusion

Satu entitas data akun hanya dapat dikunci dan dimutasi oleh satu transaksi pada satu waktu.

LOCK RETAINED

2. Hold and Wait

Transaksi 1 memegang Lock Akun A sambil menunggu pelepasan Lock Akun B dari pihak lain.

VOLUNTARY RELEASE

3. No Preemption

Kunci data tidak dapat direbut paksa; hanya dilepas ketika commit atau rollback terjadi.

DEADLY EMBRACE

4. Circular Wait

Hanya pada jalur ini transaksi saling mengunci dan terhenti permanen tanpa intervensi luar.

Deadlock terjadi ketika empat syarat Coffman hadir bersamaan: eksklusivitas, menahan lock lama, tanpa perebutan paksa, dan siklus tunggu melingkar. Hanya pada jalur ini kondisi saling sandera terkunci permanen, menuntut adanya intervensi eksternal seperti pembatalan paksa oleh database engine.

A closed circular dependency graph locks all participants until an external entity terminates one node.

Alur Deteksi Mesin Basis Data: Dari Wait-For Graph ke Abort

Bagaimana database engine mendeteksi siklus dan memilih satu transaksi sebagai korban.

WAIT QUEUE

1. Lock Contention

Transaksi meminta lock pada baris data sibuk dan dimasukkan ke dalam antrean tunggu.

DEADLOCK_TIMEOUT

2. Timeout Trigger

Setelah ambang waktu terlewati (default PG 1s), thread monitor pengecekan siklus dijalankan.

WAIT-FOR GRAPH

3. Cycle Detection

Algoritma DFS memeriksa graf dependensi transaksi untuk menemukan siklus tertutup.

ENGINE ABORT

4. Victim Selection

Mesin membatalkan transaksi korban; belum membuktikan algoritma cost-based di luar lab.

Basis data tidak memeriksa deadlock secara terus-menerus demi menghemat CPU, melainkan menunggu jeda timeout tertentu sebelum menelusuri Wait-For Graph. Simulasi lab membatalkan transaksi via context timeout, tetapi belum membuktikan algoritma pemilihan korban berbasis biaya modifikasi terkecil seperti MySQL InnoDB.

Periodic background graph analysis identifies cyclic dependencies and evicts minimal-cost victim transactions.

Pola Penguncian Naif yang Menimbulkan Deadlock

Mengakuisisi lock sesuai urutan input membiarkan arah transaksi saling bertubrukan.

```
func TransferNaive(ctx context.Context, from, to *bank.Account, amount int, delay time.Duration) error {
    if err := from.Lock(ctx); err != nil { return err }
    defer from.Unlock()
    time.Sleep(delay) // simulasi durasi transaksi internal
    if err := to.Lock(ctx); err != nil { return err }
    defer to.Unlock()
    from.Balance -= amount
    to.Balance += amount
    return nil
}
```

Fungsi TransferNaive mengunci akun pengirim lalu akun penerima sesuai urutan parameter pemanggilan. Menambahkan jeda waktu eksekusi bukan berarti memperlambat kode secara sengaja, melainkan memodelkan latensi query nyata yang membuka celah tabrakan bagi transaksi berlawanan arah.

Binding lock acquisition order directly to call arguments creates bidirectional lock acquisition hazards.

Kronologi Tabrakan: Goroutine A vs Goroutine B

Langkah demi langkah runtuhnya sistem akibat dua transaksi saling menyandera kunci.

FASE KEGAGALAN TRANSAKSI SILANG

Langkah 1: Akuisisi Parsial

Goroutine 1 berhasil mengunci Akun A; Goroutine 2 bersamaan mengunci Akun B.

FASE KEGAGALAN TRANSAKSI SILANG

Langkah 2: Saling Menunggu

Goroutine 1 mencoba mengunci Akun B (sibuk); Goroutine 2 mencoba mengunci Akun A (sibuk).

FASE KEGAGALAN TRANSAKSI SILANG

Langkah 3: Kebuntuan Mutlak

Kedua goroutine terblokir; tidak ada yang dapat melanjutkan atau melepas lock secara sukarela.

FASE KEGAGALAN TRANSAKSI SILANG

Langkah 4: Terminasi Paksa

Hanya pada jalur ini context timeout memutus rantai dan mengembalikan galat ErrDeadlock.

Ketika dua transfer berjalan simultan dengan arah berlawanan, masing-masing berhasil mendapatkan kunci pertama namun gagal memperoleh kunci kedua. Hanya pada jalur ini kebuntuan absolut terjadi, memaksa timer pembatalan mengorbankan salah satu transaksi agar yang lain dapat selesai.

Simultaneous cross-resource acquisition without preemption leads to an unresolvable terminal deadlock.

Matriks Diagnostik: Fenomena Penguncian di Database

Membedakan karakteristik teknis deadlock, lock timeout, dan lock contention biasa.

Dimensi Masalah	Normal Lock Contention	Lock Timeout	Deadlock (Circular Wait)
Kondisi Dependensi	Antrean linear satu arah	Tunggu melebihi ambang batas	Siklus tertutup saling sandera (A->B->A)
Penyelesaian Mesin	Menunggu lock dilepas	Transaksi pemohon dibatalkan	Satu transaksi dikorbankan via rollback
Kode Galat Umum	Tidak ada galat (hanya antre)	MySQL 1205 (wait timeout)	PG 40P01 / MySQL 1213 / SQLServer 1205
Akar Masalah	Beban transaksi tinggi	Transaksi lambat / long-running	Pola urutan akuisisi lock tidak konsisten
Solusi Arsitektural	Indexing & isolasi lebih rendah	Perpendek durasi transaksi	Lock Ordering konsisten & Retry Pattern

Tabel ini memetakan perbedaan fundamental antara antrean lock normal, lock timeout, dan kondisi deadlock murni. Deadlock memiliki ciri khas dependensi siklik yang tidak dapat selesai sendiri, sehingga selalu memerlukan intervensi rollback aktif dari engine basis data.

Distinguish linear queuing delays from cyclic deadlock dependencies requiring automatic abort intervention.

Tiga Lapisan Pertahanan Menghadapi Deadlock

Hierarki strategi rekayasa perangkat lunak dari pencegahan dini hingga pemulihan aktif.

STRATEGI PENANGANAN DEADLOCK BERTAHAP

1. Lock Ordering

Urutkan seluruh resource berdasarkan ID sebelum akuisisi kunci; memutus circular wait secara matematis.

STRATEGI PENANGANAN DEADLOCK BERTAHAP

2. Perpendek Transaksi

Keluarkan panggilan jaringan pihak ketiga dan kalkulasi berat dari dalam blok transaksi SQL.

STRATEGI PENANGANAN DEADLOCK BERTAHAP

3. Application Retry

Tangkap kode galat deadlock; bukan berarti pasrah, melainkan pulihkan transaksi via backoff.

Penanganan deadlock yang kokoh memadukan pencegahan dan ketahanan aplikasi. Mengimplementasikan retry otomatis bukan berarti mengabaikan pencegahan, melainkan jaring pengaman wajib untuk skenario konkurensi ekstrem yang tidak terduga oleh skema ordering.

Prevent cycles deterministically via lock ordering, minimize lock duration, and recover residual aborts via retries.

Eksekusi Test Suite dan Demo cmd/demo Lolos Bersih

Output terminal membuktikan reproduksi deadlock, prevensi ordering, dan pemulihan retry.

```
=== RUN   TestDeadlockOccurrence
--- PASS: TestDeadlockOccurrence (0.02s) [victim identified]
=== RUN   TestLockOrderingPreventsDeadlock
--- PASS: TestLockOrderingPreventsDeadlock (0.01s) [balances consistent]
=== RUN   TestRetryRecoversDeadlock
--- PASS: TestRetryRecoversDeadlock (0.02s) [both transfers complete]
=== RUN   TestTransactionDurationImpact
--- PASS: TestTransactionDurationImpact (0.05s) [long >= short]
PASS: ok  labs/18-deadlock/tests  1.715s (race detector: clean)
1. Simulating Naive Concurrent Transfers -> deadlock victim detected
2. Simulating Lock Ordered Transfers    -> ACC-1=1100, ACC-2=900 (success)
3. Simulating Retry Mechanism           -> Final Balances: ACC-1=1100, ACC-2=900
```

Verifikasi terminal membuktikan seluruh skenario uji lolos tanpa race condition di bawah pengawasan Go race detector. Demonstrasi CLI mengonfirmasi bahwa pendekatan naif menghasilkan deadlock victim, sedangkan pengurutan kunci dan mekanisme retry menyelesaikan mutasi dengan konsisten.

Executable test proof confirms deterministic reproduction, mathematical prevention, and transparent recovery.

engineering/03-execution-result.md:14-61

Batasan Bukti Lab: Yang Dibuktikan vs Belum

Menegaskan batasan simulasi in-memory terhadap kompleksitas database engine riil.

BATAS PEMBUKTIAN SISTEM LAB 18

YANG DIBUKTIKAN

Reproduksi circular wait, keampuhan pengurutan ID kunci, dan pemulihan saldo via retry loop.

BATAS PEMBUKTIAN SISTEM LAB 18

YANG TIDAK DIBUKTIKAN

Mesin graph parser WFG riil, multi-table foreign key cascades, dan gap locking di database.

BATAS PEMBUKTIAN SISTEM LAB 18

BATAS KLAIM EMPIRIS

Lab belum membuktikan ketahanan sistem pada skala 50.000 TPS dengan distribusi kunci acak.

Lab ini membuktikan mekanika dasar deadlock dan validitas teknik pengurutan kunci menggunakan channel in-memory. Pengujian ini belum membuktikan fenomena kompleks seperti gap locking pada indeks database riil atau perilaku sistem pada beban puluhan ribu TPS.

Strictly separate application-level synchronization models from relational storage engine locking internals.

content/01-content-brief.md:12-16

Kompromi Rekayasa: Lock Ordering vs Application Retry

Mengevaluasi konsekuensi operasional antara pengurutan ketat dan toleransi pembatalan.

ANALISIS PERBANDINGAN TRADE-OFF

Lock Ordering: Disiplin Kode

Bebas deadlock sepenuhnya, namun mewajibkan kesepakatan urutan global di seluruh tim pengembang.

ANALISIS PERBANDINGAN TRADE-OFF

Retry: Beban Komputasi

Mudah dipasang di wrapper, namun memboroskan CPU dan I/O jika terjadi abort berulang-ulang.

ANALISIS PERBANDINGAN TRADE-OFF

Tidak Otomatis Cepat

Menambah retry tidak otomatis meningkatkan throughput bila contention pada satu baris data terlalu pekat.

Penerapan lock ordering memberikan kepastian absolut namun menuntut disiplin penulisan query yang seragam di seluruh repository. Menambahkan retry logic tidak otomatis membuat sistem berkinerja tinggi jika sumber daya yang diperebutkan mengalami saturasi ekstrem.

Choose between upfront developmental discipline via ordering or operational compute overhead via retries.

Model Sinkronisasi Akun Berbasis Channel dan Timeout

Menggunakan channel kapasitas 1 sebagai mutex yang peka terhadap pembatalan context.

```
type Account struct {
    ID      string
    Balance int
    ch      chan struct{} // channel buffer 1 sebagai token lock
}
func (a *Account) Lock(ctx context.Context) error {
    select {
    case <-a.ch: return nil
    case <-ctx.Done(): return ErrDeadlock // representasi deadlock abort
    }
}
```

Model Account memanfaatkan buffered channel 1 slot untuk mengendalikan hak akses mutasi saldo. Simulasi ini tidak sama dengan locking row-level di PostgreSQL yang memiliki status lock modes bertingkat, namun secara elegan merefleksikan pembatalan transaksi akibat timeout eksternal.

Channel token acquisition coupled with context cancellation models interruptible resource acquisition.

internal/bank/account.go:10–33

Transformasi: Dari Tabrakan Acak ke Eksekusi Teratur

Pergeseran dari pemanggilan naif yang rentan menuju orkestrasi penguncian deterministik.

KOMPARASI KONDISI SISTEM TRANSAKSI

Sebelum: Urutan Dinamis

Kunci diambil mengikuti urutan parameter fungsi; transfer dua arah saling menjebak dalam circular wait.

KOMPARASI KONDISI SISTEM TRANSAKSI

Sesudah: Leksikografis

ID akun diurutkan secara konsisten; kedua proses antre secara teratur pada lock pertama yang sama.

KOMPARASI KONDISI SISTEM TRANSAKSI

Bukan Berarti Lambat

Mengantre pada satu kunci bukan berarti menurunkan throughput; sistem terhindar dari biaya rollback.

Sebelum standarisasi urutan, dua transaksi yang bersilangan akan saling menunggu dan berujung pada pembatalan sepihak. Mengantrekan transaksi pada kunci pertama bukan berarti memperlambat sistem, melainkan menyelamatkan CPU dari pemborosan rollback dan retry.

Convert circular blocking hazards into deterministic linear queues via strict lexical key sorting.

Pencegahan Deterministik: Pengurutan Kunci Berdasarkan ID

Menjamin bahwa semua proses selalu meminta lock dengan urutan seragam tanpa siklus.

```
func TransferOrdered(ctx context.Context, acc1, acc2 *bank.Account, amount int, delay time.Duration) error {
    first, second := acc1, acc2
    if acc1.ID > acc2.ID { first, second = acc2, acc1 }
    if err := first.Lock(ctx); err != nil { return err }
    defer first.Unlock()
    time.Sleep(delay)
    if err := second.Lock(ctx); err != nil { return err }
    defer second.Unlock()
    acc1.Balance -= amount; acc2.Balance += amount
    return nil
}
```

Dengan membandingkan identifier string secara leksikografis, entitas dengan ID lebih kecil selalu dikunci terlebih dahulu. Logika pencegahan ini cakupannya mengikuti test konkurensi lab, menjamin bahwa circular wait mustahil terbentuk karena seluruh goroutine mengantre pada kunci pertama yang sama.

Global monotonic resource acquisition eliminates cycles across all concurrent transaction paths.

internal/transfer/transfer.go:28–50

Verifikasi Test: Membuktikan Terjadinya Deadlock Victim

Dua goroutine bersaing memicu tabrakan deterministik pada penguncian naif.

```
func TestDeadlockOccurrence(t *testing.T) {
    // Goroutine 1 mentransfer A -> B (delay 10ms, timeout 20ms)
    go func() { errs[0] = transfer.TransferNaive(ctx1, accA, accB, 10, 10*time.Millisecond) }()
    // Goroutine 2 bersamaan mentransfer B -> A (delay 10ms, timeout 20ms)
    go func() { errs[1] = transfer.TransferNaive(ctx2, accB, accA, 10, 10*time.Millisecond) }()
    wg.Wait()
    hasDeadlock := (errs[0] == bank.ErrDeadlock) || (errs[1] == bank.ErrDeadlock)
    if !hasDeadlock { t.Fatalf("expected at least one deadlock victim") }
}
```

Unit test membuktikan secara deterministik bahwa transfer silang tanpa pengurutan selalu mengorbankan minimal satu transaksi. Menjalankan dua goroutine paralel tidak otomatis memicu deadlock tanpa delay internal yang memadai untuk membuka jendela kontensi.

Reproducing deadlocks requires synchronized contention windows where resources are held during secondary requests.

tests/transfer_test.go:13–41

Verifikasi Test: Lock Ordering Menuntaskan Semua Transaksi

Kedua transfer konkuren berhasil diselesaikan dengan saldo akhir yang konsisten sempurna.

```
func TestLockOrderingPreventsDeadlock(t *testing.T) {
    go func() { errs[0] = transfer.TransferOrdered(ctx1, accA, accB, 10, 5*time.Millisecond) }()
    go func() { errs[1] = transfer.TransferOrdered(ctx2, accB, accA, 10, 5*time.Millisecond) }()
    wg.Wait()
    if errs[0] != nil || errs[1] != nil { t.Fatalf("expected both to succeed") }
    if accA.Balance != 100 || accB.Balance != 100 {
        t.Fatalf("unexpected balance: A=%d, B=%d", accA.Balance, accB.Balance)
    }
}
```

Pengujian membuktikan bahwa kedua transaksi bolak-balik berhasil dieksekusi tanpa error sama sekali saat menggunakan TransferOrdered. Hanya pada jalur ini konsistensi saldo terjaga utuh (100 dan 100) karena transaksi saling mengantre secara tertib alih-alih saling mematikan.

Enforcing a single acquisition order linearizes resource contention, preserving full transaction integrity.

tests/transfer_test.go:43–74

Pemulihan Transparan: Penanganan Abort dengan Retry

Membungkus eksekusi dengan batas timeout percobaan dan jeda backoff terencana.

```
func TransferWithRetry(ctx context.Context, from, to *bank.Account, amount int, delay time.Duration, maxRetries int) error {
    for i := 0; i < maxRetries; i++ {
        attemptCtx, cancel := context.WithTimeout(ctx, 10*time.Millisecond)
        err := TransferNaive(attemptCtx, from, to, amount, delay)
        cancel()
        if err == nil { return nil }
        if err != bank.ErrDeadlock && err != context.DeadlineExceeded { return err }
        time.Sleep(2 * time.Millisecond) // backoff sebelum mencoba kembali
    }
    return bank.ErrDeadlock
}
```

Fungsi TransferWithRetry menangkap galat pembatalan deadlock dan mencoba kembali transaksi secara transparan hingga batas maksimal. Menggunakan fixed backoff 2ms bukan berarti pola ini ideal untuk produksi; implementasi ini adalah penyederhanaan terencana yang membutuhkan jitter pada skala besar.

Catch transient concurrency aborts and replay idempotent transactions following brief backoff intervals.

Dampak Durasi: Transaksi Panjang Memperbesar Risiko

Membandingkan frekuensi deadlock antara transaksi dengan delay versus instan.

```
func TestTransactionDurationImpact(t *testing.T) {
    longDeadlocks := countDeadlocks(5*time.Millisecond, 10)
    shortDeadlocks := countDeadlocks(0, 10)
    if longDeadlocks < shortDeadlocks {
        t.Fatalf("expected long duration to have >= deadlocks: long=%d, short=%d",
                 longDeadlocks, shortDeadlocks)
    }
}
```

Pengujian empiris membuktikan bahwa transaksi yang menahan lock selama 5 milidetik menghasilkan frekuensi deadlock yang lebih tinggi atau sama dibandingkan transaksi tanpa delay. Pengamatan ini cakupannya mengikuti test lab, memvalidasi prinsip bahwa memperpanjang waktu penahanan lock memperbesar kemungkinan tabrakan.

Deadlock probability scales proportionally with the duration locks are held open by active transactions.

tests/transfer_test.go:105–145

Checklist Proteksi Deadlock di Lingkungan Produksi

Panduan komprehensif sebelum merilis kode transaksi konkuren ke basis data.

- ✓ Terapkan Lock Ordering seragam pada seluruh mutasi multi-tabel di seluruh repositori.
- ✓ Lakukan ORDER BY primary_key ASC sebelum query UPDATE batch atau SELECT FOR UPDATE.
- ✓ Keluarkan panggilan HTTP eksternal dan komputasi berat dari dalam transaksi database.
- ✓ Tangkap kode galat deadlock vendor (Postgres 40P01, MySQL 1213, SQL Server 1205).
- ✓ Terapkan exponential backoff dengan randomized jitter pada lapisan retry aplikasi.
- ✓ Pastikan seluruh operasi yang di-retry bersifat idempotent untuk mencegah mutasi ganda.

Enforce strict operational criteria to design, isolate, and recover concurrent database transactions.

Anti-Pattern Umum dalam Penanganan Transaksi

Kesalahan desain yang sering memicu deadlock persisten pada sistem produksi.

POLA BERBAHAYA YANG HARUS DIHINDARI

I/O Eksternal di Transaksi

Memanggil API pihak ketiga atau menunggu I/O disk saat koneksi database masih memegang row lock.

POLA BERBAHAYA YANG HARUS DIHINDARI

Batch Update Acak

Mengupdate ratusan baris data dari array memori tanpa mengurutkan ID terlebih dahulu.

POLA BERBAHAYA YANG HARUS DIHINDARI

Bukan Berarti Aman

Memasang retry tanpa idempotensi bukan berarti aman, melainkan memicu mutasi saldo ganda.

Menahan koneksi transaksi database sambil menunggu respons API pihak ketiga adalah penyebab paling umum lonjakan deadlock di produksi. Memasang wrapper retry tanpa memastikan idempotensi mutasi bukan berarti sistem menjadi andal, melainkan berisiko mencatatkan perubahan saldo ganda.

Avoid external I/O within transaction boundaries and never retry non-idempotent business operations.

Reproduksi Mandiri di Mesin Lokal Menggunakan Go

Verifikasi mandiri simulasi deadlock tanpa ketergantungan modul eksternal.

```
cd labs/18-deadlock
go test -v ./...    # verifikasi 4 pengujian deadlock, ordering, retry & durasi
go test -race ./... # pembuktian sinkronisasi konkuren bebas data race
go run ./cmd/demo   # eksekusi demonstrasi CLI interaktif skenario 1, 2, dan 3
```

Seluruh simulasi dibangun murni menggunakan pustaka standar Go tanpa modul pihak ketiga. Eksperimen lokal ini memvalidasi mekanika deadlock di memori, namun production masih perlu mengonfigurasi parameter database nyata seperti deadlock_timeout dan log_lock_waits.

Zero-dependency concurrency simulations provide fast, deterministic validation of distributed failure modes.

README.md:14-34

Prinsip Inti: Deadlock adalah Keniscayaan Konkurensi

Pondasi arsitektur tangguh bertumpu pada pencegahan terstruktur dan toleransi kegagalan.

TIGA PILAR KETAHANAN KONKURENSI SISTEM

Disiplin Urutan Global

Jadikan pengurutan kunci sebagai standar baku query di seluruh lapisan arsitektur microservices.

TIGA PILAR KETAHANAN KONKURENSI SISTEM

Keringkasan Transaksi

Pangkas waktu hidup transaksi seminimal mungkin; lock hanya saat mutasi siap dieksekusi.

TIGA PILAR KETAHANAN KONKURENSI SISTEM

Toleransi Transparan

Rancang aplikasi agar tidak otomatis panik saat menerima abort; pulihkan via retry transparan.

Deadlock tidak boleh dipandang sebagai aib atau bug fatal, melainkan mekanisme perlindungan integritas data dari basis data. Merancang sistem transaksional tidak otomatis membebaskan kita dari kontensi; sistem yang tangguh mencegah siklus sedini mungkin dan memulihkan korban dengan anggun.

Design for deterministic lock ordering upfront, minimize critical sections, and treat abort recovery as standard operating procedure.

content/05-key-takeaways.md:1-8

Deadlock bukan kegagalan basis data, melainkan alarm bahwa aplikasi Anda mengunci sumber daya secara acak.

Deadlocks are structural alarms indicating uncoordinated concurrent resource access patterns.

Software Engineering Lab 18 – Deadlock di Sistem Konkuren

internal/bank/account.go • internal/transfer/transfer.go
cmd/demo/main.go • tests/transfer_test.go
engineering/01-design.md • content/02-master-draft.md

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/18-deadlock`

Lukman / lukman-ss

Commit: e76d13f • Status: APPROVED
Test passing: 4 test suites (DeadlockOccurrence, LockOrdering, RetryRecovery, DurationImpact)
Race detector: Clean (go test -race ./... PASS in 1.715s)
Zero external dependencies: 100% Go standard library.